

# SpecuNode

Speculative execution for agent graphs, with a store buffer: what happens when an AI agent runs ahead of its own model, and how to make sure nothing it guessed reaches the world.

**Every number in this document is read from a committed file in `bench/results/`.** Nothing here is typed in by hand. A section whose results file is missing says so rather than being omitted.

## 1. How much speculation a real corpus exposes

Corpus: `nebius/SWE-rebench-openhands-trajectories` (huggingface), 300 trajectories, 19484 tool calls. Effect classes come from a committed hand-written table per tool name; anything absent from it is a `WRITE`, exactly as in the runtime.

| Measure                             | Mean [95% CI]           |
|-------------------------------------|-------------------------|
| Reads - speculable under any policy | 0.0445 [0.0423, 0.0469] |
| Calls opening a new model turn      | 1.0000 [1.0000, 1.0000] |
| Args referencing a prior result     | 0.1957 [0.1875, 0.2040] |
| PASTE speculable span (calls)       | 0.0462 [0.0438, 0.0487] |
| Steps PASTE skips, SpecuNode stages | 0.9555 [0.9531, 0.9577] |
| SpecuNode span past a write (calls) | 0.0000 [0.0000, 0.0000] |
| Model turn consuming a write result | 0.9547 [0.9523, 0.9569] |

**On this corpus, running ahead past a write buys nothing at all.** The span is 0.0000, with a bootstrap interval that does not move off zero. Every tool call in these trajectories opens a new model turn, and a staged write blocks the next turn because that turn would have to contain a placeholder where the real result belongs. 95.5% of the corpus is that shape.

The half that is not zero: PASTE excludes a tool with side effects from speculation entirely, so it can speculate on 4.5% of steps. SpecuNode stages a write instead of refusing it, covering the other 95.5%. That is an upper bound on opportunity, not a speedup, realisable only where the predictor is right. The nearest measured proxy is signature accuracy at 53.5% top-1, which compares tool names and argument keys but not argument values, and is therefore an upper bound on acceptance rather than a measurement of it top-1. Neither number means anything alone.

## The acceptance rate, measured

The signature figure is an upper bound; this is the quantity itself. The tier-1 predictor, graded by the runtime's own gate on exact canonical equality of argument values, over all 300 trajectories joined to their argument values, leave-one-trajectory-out. **within\_turn** is the policy the runtime runs; **across\_turns** is what carrying a guess into the next model turn would be worth.

| Measure                        | within_turn             | across_turns            |
|--------------------------------|-------------------------|-------------------------|
| Steps graded                   | 19184                   | 19184                   |
| Guesses offered                | 0.9533                  | 0.9794                  |
| Acceptance rate, pooled        | 0.0000                  | 0.0002                  |
| Acceptance rate, by trajectory | 0.0000 [0.0000, 0.0000] | 0.0002 [0.0000, 0.0004] |

|                             |        |        |
|-----------------------------|--------|--------|
| Confirmed guesses           | 0      | 3      |
| Signature top-1, same steps | 0.0000 | 0.5339 |
| Squashed at a turn boundary | 18288  | 0      |

**The tier-1 acceptance rate on this corpus is 0.0002 at best and 0.0000 under the runtime's own policy.** The index knows which tool comes next 53.4% of the time; the gate needs the exact command, path or thought. Every argument value of a call has already appeared earlier at 9.8% of steps, which is the ceiling for any predictor that copies values out of history, and this one is far below it. Anything above the ceiling needs a predictor that generates values, and that has not been measured.

## 2. What beats it

| #    | Strategy                             | Outcome  | Measured                                             |
|------|--------------------------------------|----------|------------------------------------------------------|
| 7.1  | misdeclared tool (READ that writes)  | beats it | effects_from_squashed_branch=1; leak_rate=1.0        |
| 7.2  | read with upstream side effects      | beats it | reads_from_squashed_branches=5; reads_issued=10      |
| 7.3  | stale reads under contention         | beats it | detected_stale=1; reads=2; undetectable_fraction=0.5 |
| 7.4  | duplicate delivery of a non-idempote | beats it | delivered=2; duplicates=1; intended=1                |
| 7.5  | return-value laundering              | beats it | cases=9; caught=6; miss_rate=0.333                   |
| 7.6  | prompt-injected tool call            | beats it | dispatched_when_only_the_drafter_predicted_it=0; dis |
| 7.7  | drafter poisoning                    | held     | alpha_window=4; charges_staged_then_discarded=4; con |
| 7.8  | replay under model drift             | held     | both_caught_at_step_0=1; first_divergence_step_after |
| 7.9  | staging an irreversible effect       | beats it | barrier_by_default=1; staged_when_enabled=1          |
| 7.10 | asynchronous side effect behind a RE | beats it | effects_landing_after_the_squash=1; leaked_effects_p |

## 3. Demo 1 - speculation without a store buffer

| Runtime        | Runs | Mispredictions | Reached world | From squashed branches |
|----------------|------|----------------|---------------|------------------------|
| naive-parallel | 50   | 10             | 60            | 10                     |
| specunode      | 50   | 10             | 50            | 0                      |
| sequential     | 50   | 0              | 50            | 0                      |

The last column, and the fact that the second and third rows agree.

## 4. Invariants under load, and what the runtime costs

| Matrix      | Runs | Leaks | Duplicates | Equivalence failures |
|-------------|------|-------|------------|----------------------|
| Chaos       | 12   | 0     | 0          | 0                    |
| Concurrency | 20   | 0     | 0          | 0                    |

Journaling and classification cost 6.645 ms per step (19.934 ms per run over 3 steps). That is 87.7% of wall clock here, and the percentage is the misleading half: a scripted model answers instantly, so this is the worst case for the ratio.

## 5. Wall-clock latency

**Not measured.** The online latency benchmark calls a real target model and needs an API key and a spend cap. No wall-clock figure appears anywhere in this report or in the repository, because none has been produced.